

```
import pandas as pd
df = pd.read_excel("/content/Super Market Sales Analysis.xlsx")

df.head()
```

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	Total	Date	Time	Payment	cogs	pe
0	750-67-8428	A	Yangon	Member	Female	Health and beauty	74.69	7	26.1415	548.9715	2019-01-05	13:08:00	Ewallet	522.83	
1	226-31-3081	C	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200	80.2200	2019-03-08	10:29:00	Cash	76.40	
2	631-41-3108	A	Yangon	Normal	Male	Home and lifestyle	46.33	7	16.2155	340.5255	2019-03-03	13:23:00	Credit card	324.31	
3	123-19-1176	A	Yangon	Member	Male	Health and beauty	58.22	8	23.2880	489.0480	2019-01-27	20:33:00	Ewallet	465.76	
4	373-73-7910	A	Yangon	Normal	Male	Sports and travel	86.31	7	30.2085	634.3785	2019-02-08	10:37:00	Ewallet	604.17	

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Invoice ID                            1000 non-null   object
1   Branch                              1000 non-null   object
2   City                                1000 non-null   object
3   Customer type                        1000 non-null   object
4   Gender                              1000 non-null   object
5   Product line                         1000 non-null   object
6   Unit price                           1000 non-null   float64
7   Quantity                             1000 non-null   int64
8   Tax 5%                              1000 non-null   float64
9   Total                               1000 non-null   float64
10  Date                                 1000 non-null   datetime64[ns]
11  Time                                1000 non-null   object
12  Payment                             1000 non-null   object
13  cogs                                 1000 non-null   float64
14  gross margin percentage              1000 non-null   float64
15  gross income                         1000 non-null   float64
16  Rating                              1000 non-null   float64
dtypes: datetime64[ns](1), float64(7), int64(1), object(8)
memory usage: 132.9+ KB
```

```
df.describe()
```

	Unit price	Quantity	Tax 5%	Total	Date	cogs	gross margin percentage	gross income	Rating	
count	1000.000000	1000.000000	1000.000000	1000.000000	1000	1000.00000	1.000000e+03	1000.000000	1000.00000	
mean	55.672130	5.510000	15.379369	322.966749	2019-02-14 00:05:45.600000	307.58738	4.761905e+00	15.379369	6.97270	
min	10.080000	1.000000	0.508500	10.678500	2019-01-01 00:00:00	10.17000	4.761905e+00	0.508500	4.00000	
25%	32.875000	3.000000	5.924875	124.422375	2019-01-24 00:00:00	118.49750	4.761905e+00	5.924875	5.50000	
50%	55.230000	5.000000	12.088000	253.848000	2019-02-13 00:00:00	241.76000	4.761905e+00	12.088000	7.00000	

```
df.value_counts()
```

Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	Total	Date	Time	Payment	cogs	per
898-04-2717	A	Yangon	Normal	Male	Fashion accessories	76.40	9	34.3800	721.9800	2019-03-19	15:49:00	Ewallet	687.60	4.7
101-17-6199	A	Yangon	Normal	Male	Food and beverages	45.79	7	16.0265	336.5565	2019-03-13	19:44:00	Credit card	320.53	4.7
101-81-4070	C	Naypyitaw	Member	Female	Health and beauty	62.82	2	6.2820	131.9220	2019-01-17	12:36:00	Ewallet	125.64	4.7
102-06-2002	C	Naypyitaw	Member	Male	Sports and travel	25.25	5	6.3125	132.5625	2019-03-20	17:52:00	Cash	126.25	4.7
102-77-2261	C	Naypyitaw	Member	Male	Health and beauty	65.31	7	22.8585	480.0285	2019-03-05	18:02:00	Credit card	457.17	4.7
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
115-38-7388	C	Naypyitaw	Member	Female	Fashion accessories	10.18	8	4.0720	85.5120	2019-03-30	12:51:00	Credit card	81.44	4.7
114-35-5271	B	Mandalay	Normal	Female	Electronic accessories	57.91	8	23.1640	486.4440	2019-02-07	15:06:00	Cash	463.28	4.7
110-48-7033	B	Mandalay	Member	Male	Fashion accessories	32.62	4	6.5240	137.0040	2019-01-29	14:12:00	Cash	130.48	4.7
110-05-6330	C	Naypyitaw	Normal	Female	Food and beverages	39.43	6	11.8290	248.4090	2019-03-25	20:18:00	Credit card	236.58	4.7
109-86-4363	B	Mandalay	Member	Female	Sports and travel	60.08	7	21.0280	441.5880	2019-02-14	11:36:00	Credit card	420.56	4.7

1000 rows × 1 columns

dtype: int64

df.isnull().sum()

	0
Invoice ID	0
Branch	0
City	0
Customer type	0
Gender	0
Product line	0
Unit price	0
Quantity	0
Tax 5%	0
Total	0
Date	0
Time	0
Payment	0
cogs	0
gross margin percentage	0
gross income	0
Rating	0

dtype: int64

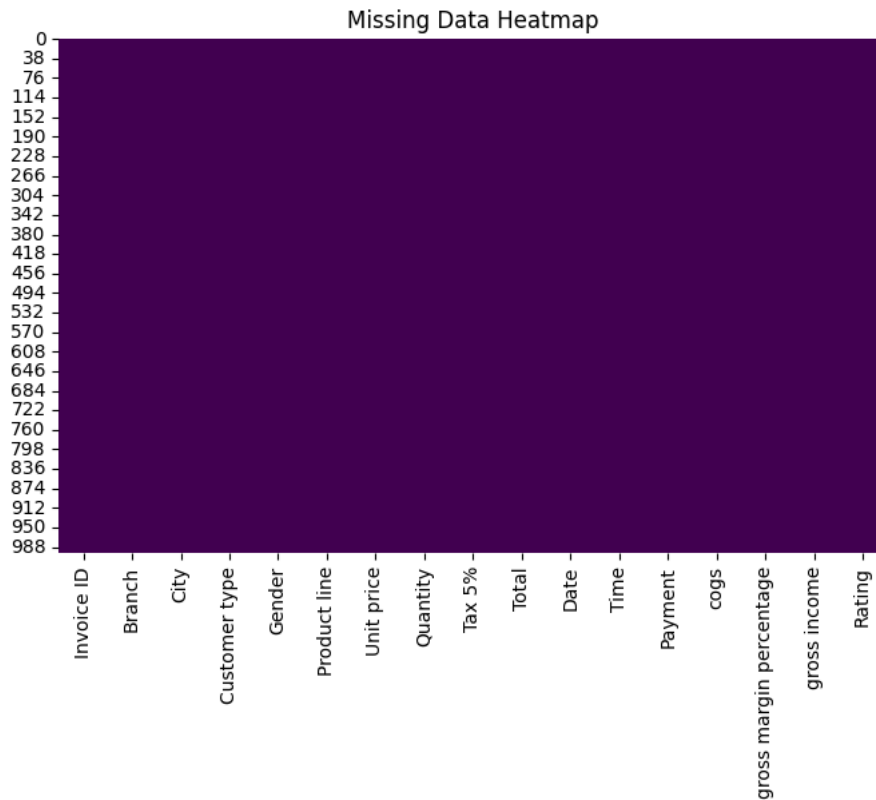
```
import seaborn as sns
import matplotlib.pyplot as plt
```

plt.figure(figsize=(8, 5))

```

sns.heatmap(df.isnull(), cbar=False, cmap="viridis")
plt.title("Missing Data Heatmap")
plt.show()

```

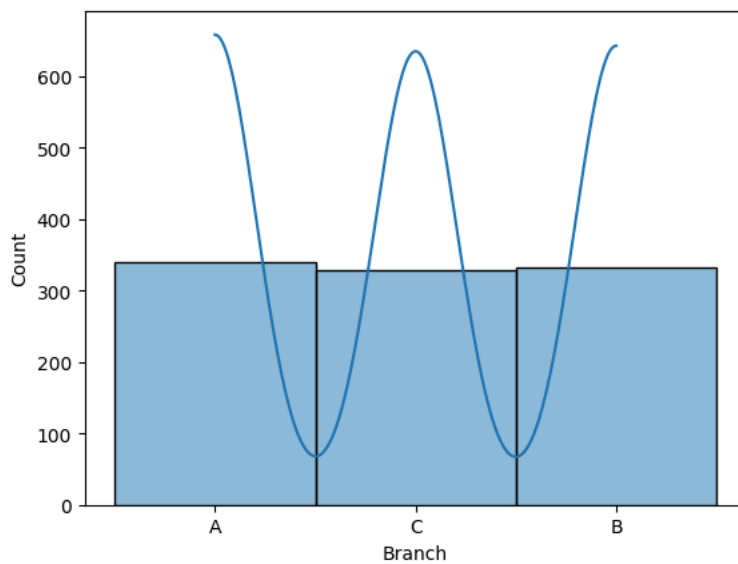


```

# Histplot
import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(df["Branch"], kde=True)
plt.show()

```



```

import seaborn as sns
import numpy as np

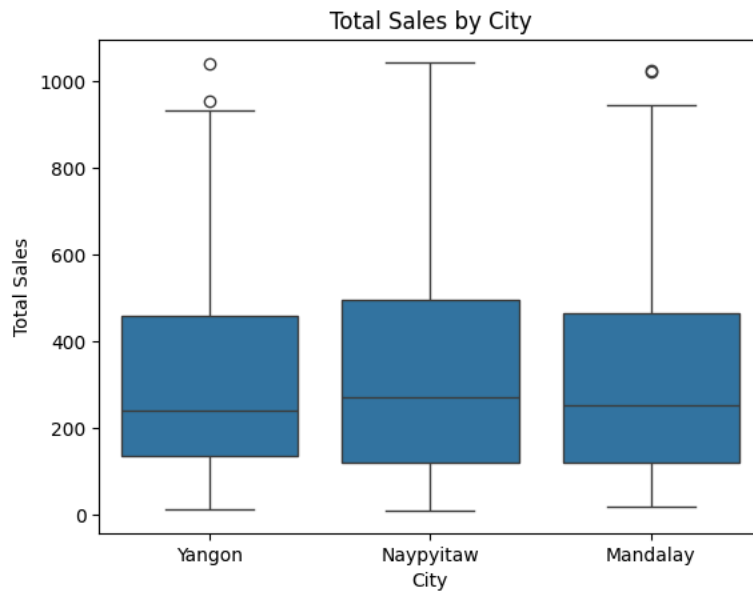
```

```

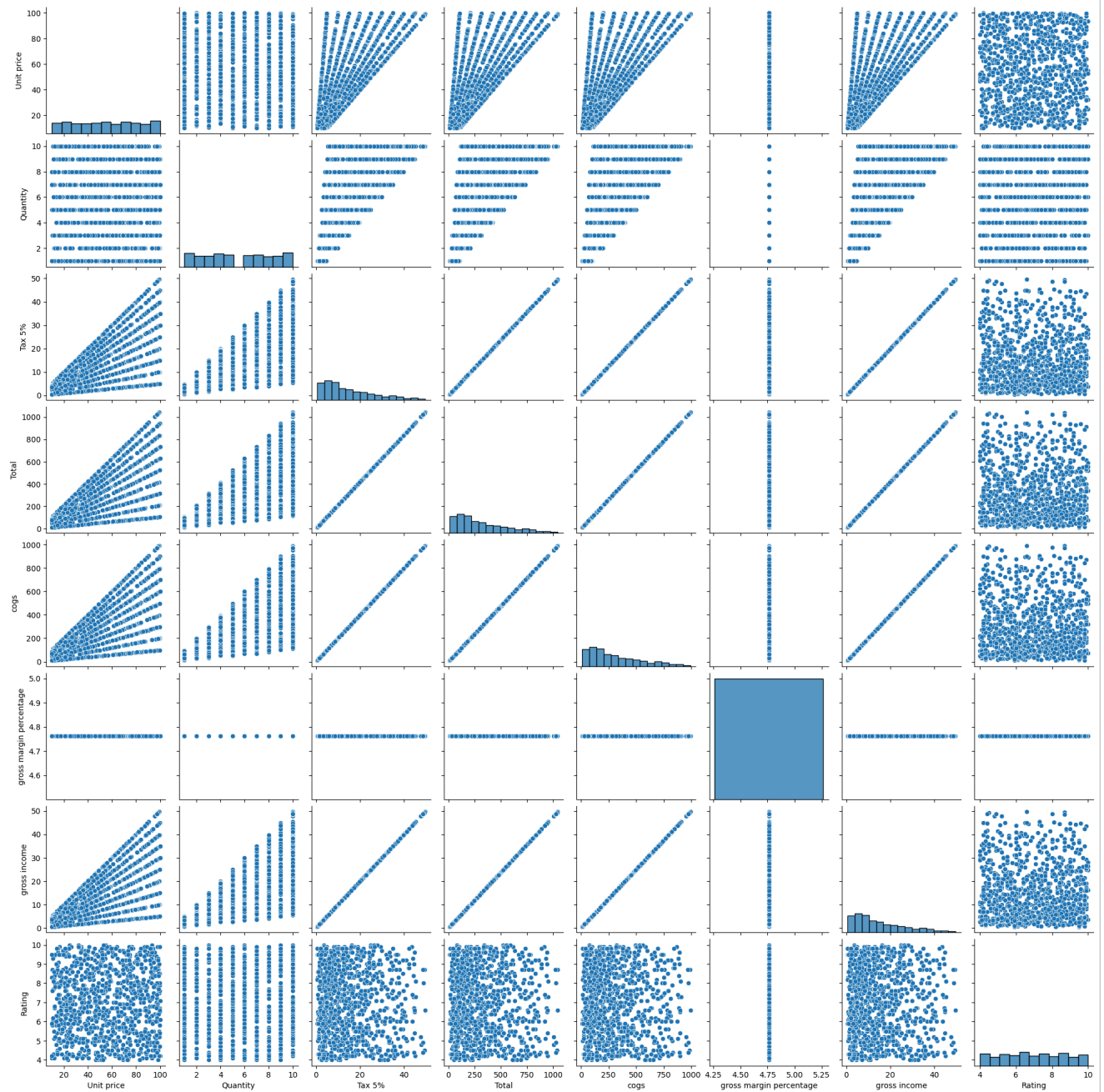
# Boxplot
sns.boxplot(x='City', y='Total', data=df,)
plt.title('Total Sales by City')
plt.xlabel('City')

```

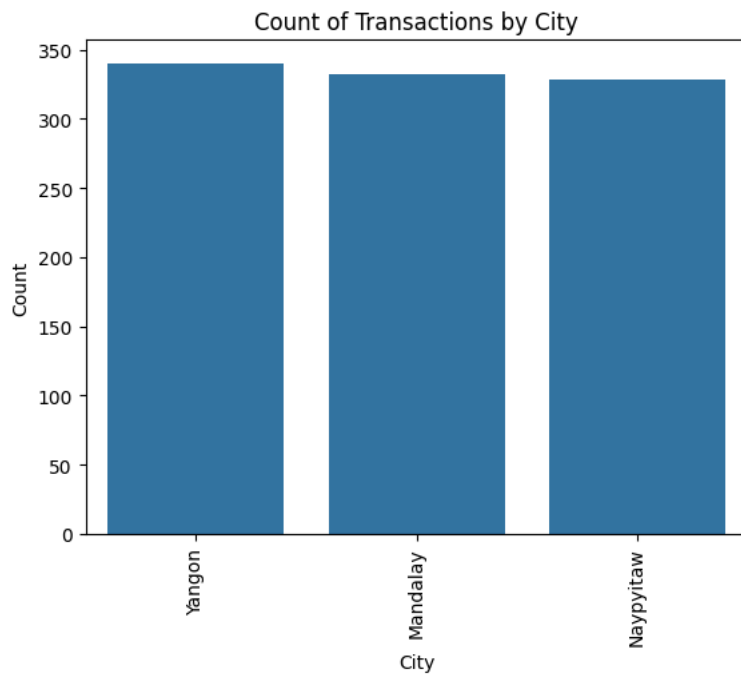
```
plt.ylabel('Total Sales')  
plt.show()
```



```
#Pairplot  
sns.pairplot(df)  
plt.show()
```



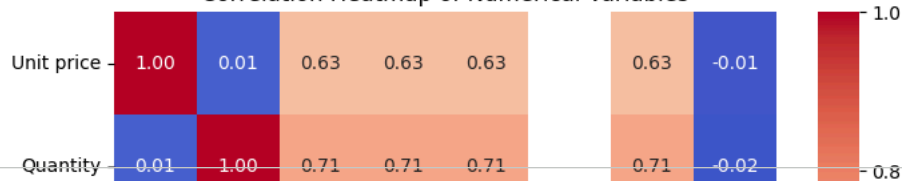
```
# Countplot
sns.countplot(data=df, x='City', order=df['City'].value_counts().index)
plt.xticks(rotation=90)
plt.title('Count of Transactions by City')
plt.xlabel('City')
plt.ylabel('Count')
plt.show()
```



```
# Calculate the correlation matrix for numerical columns
numerical_df = df.select_dtypes(include=['number'])
corr_matrix = numerical_df.corr()
```

```
# Create a heatmap
plt.figure(figsize=(8, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Heatmap of Numerical Variables')
plt.show()
```

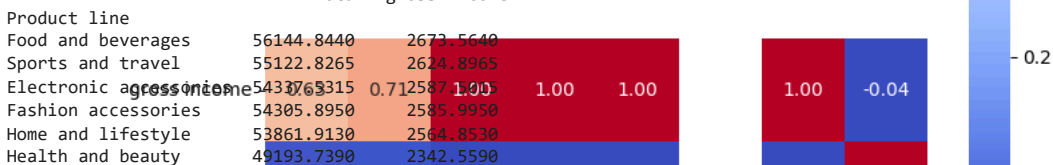
Correlation Heatmap of Numerical Variables



```
category_sales = df.groupby('Product line')[['Total', 'gross income']].sum().sort_values(by='Total', ascending=False)
print("Total Sales and Gross Income by Product Line:\n", category_sales)
```

```
# Optional: Visualize the results
category_sales.plot(kind='bar', figsize=(8, 5))
plt.title('Total Sales and Gross Income by Product Line')
plt.ylabel('Amount')
plt.xlabel('Product Line')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

Total Sales and Gross Income by Product Line:
gross margin percentage - Total gross income



Total Sales and Gross Income by Product Line

